ADR-MT-001 | Multitenancy con RLS

Plataforma multioperador de distribución de última milla

Estado	Aceptada como estrategia; motor de BD pendiente de selección.
Fecha	4 de octubre de 2026
Responsables	José Gabriel Hernández y Franco Fernández

1. Contexto y restricciones

La plataforma debe atender desde un único despliegue a varios operadores logísticos. Cada operador es un tenant con usuarios, marca y reglas propias; dentro de él, cada comercio solo accede a sus envíos. Un mismo comercio puede operar con más de un operador. El enunciado exige aislamiento comprobable tanto en lecturas como en escrituras y una demostración con al menos dos operadores de configuración distinta.

El equipo dispone de recursos institucionales limitados y no puede presupuestar varios servidores. Busca una sola base de datos para reducir infraestructura y administración, sin depender exclusivamente de los controles de la API para impedir consultas cruzadas.

2. Decisión adoptada

Se utilizará una base de datos y un esquema compartidos, con separación por fila y políticas nativas de Row-Level Security (RLS). RLS es el mecanismo de defensa en la BD para esta estrategia de tenencia; no sustituye la autenticación ni la autorización de la API.

- Las entidades de negocio compartidas incluirán OperadorId; las que pertenecen a un comercio incluirán además un vínculo de comercio dentro del operador. La autorización se evaluará por el par operador-comercio, porque un comercio puede relacionarse con varios operadores.
- La API .NET resolverá el operador y el comercio desde identidad y relaciones verificadas en el servidor. Blazor y MAUI consumirán esa API; un identificador enviado por el cliente no será autoridad suficiente para cambiar de tenant.
- EF Core aplicará filtros globales y validaciones de escritura. La BD aplicará políticas RLS para impedir lecturas y escrituras fuera del contexto autorizado, incluso si una consulta omite por error el filtro de aplicación.
- La API y el worker usarán un rol de BD con permisos mínimos. Antes de cada operación establecerán el contexto de operador y, cuando corresponda, de comercio; sin contexto válido el acceso a datos de tenant se denegará. La asignación y limpieza del contexto se verificará también con conexiones reutilizadas.

La sintaxis de políticas y contexto depende del motor elegido. La decisión sobre PostgreSQL, SQL Server u otro motor con RLS nativo se cerrará en la propuesta de stack; esta ADR no presupone un proveedor concreto.

3. Alternativas consideradas y descarte

Alternativa	Motivo del descarte
Base compartida con filtros solo en API/EF Core	Menor trabajo inicial, pero una consulta que omita el filtro podría leer datos ajenos. Se descarta porque el equipo quiere una barrera adicional en la BD.
Esquema por operador dentro de una BD	Aísla tablas por esquema, pero multiplica la gestión de modelos y migraciones. EF Core no lo admite directamente como estrategia multitenant. Se descarta por complejidad operativa.
Base de datos por operador o por comercio	Da mayor separación, pero aumenta aprovisionamiento, migraciones, respaldo y operación. Varias BD podrían vivir en un solo servidor; la limitación de servidores no la vuelve imposible por sí sola. Se descarta por el costo y mantenimiento previstos para el equipo.
Modelo híbrido	Mezcla tenants en BD compartida y dedicadas. Agrega enrutamiento y dos modelos operativos; no aporta valor suficiente para el alcance y recursos actuales.

4. Consecuencias

- Beneficios: un solo almacén transaccional simplifica el entorno; la BD agrega una barrera cuando un acceso conserva el rol de aplicación pero omite una validación o filtro de la API.
- Costos y dificultades: habrá que mantener políticas RLS en cada tabla afectada, incluirlas en migraciones, indexar las claves de tenant y medir su impacto en consultas. API, worker, reportes, seguimiento público y tareas administrativas requieren rutas de acceso definidas.
- Límite de seguridad: RLS no protege frente a cuentas de BD capaces de eludir políticas ni frente a un contexto de tenant falsificado por código comprometido. El rol de ejecución no tendrá esos privilegios; las credenciales de migración y administración estarán separadas.
- Flexibilidad futura: separar más adelante a un operador en su propia BD exigirá extracción de datos, cambios de enrutamiento y validación de las integraciones; esta decisión favorece el costo actual sobre esa facilidad futura.

Evidencia de cumplimiento prevista

Pruebas automatizadas con dos operadores de configuración distinta y comercios relacionados: lectura y escritura cruzadas rechazadas desde la API y con el rol de ejecución de la BD; ausencia de contexto denegada; cambio de tenant entre conexiones reutilizadas sin fuga; worker y consultas públicas limitados a su ámbito. Las políticas, migraciones y datos de inicialización deberán recrearse con Docker Compose y en el despliegue remoto.

Referencias

UTEC, Laboratorio .NET 2026, §§3.3, 6.5 y 8.2.

Microsoft Learn, Multi-tenancy - EF Core y Global Query Filters.

Documentación oficial: PostgreSQL Row Security Policies; SQL Server Row-Level Security.